Software Engineering Lab - Lab 7

Fuzzing for Input Validation Bugs

Arjun Srikanth | PES1UG24AM920 | Section H
Assigned program - Income Tax Parser

1. Original Application Execution

The original faculty program normalizes the sample PAN, then stops with ValueError when the amount is not money. The failure occurs before the tax slab prompt.

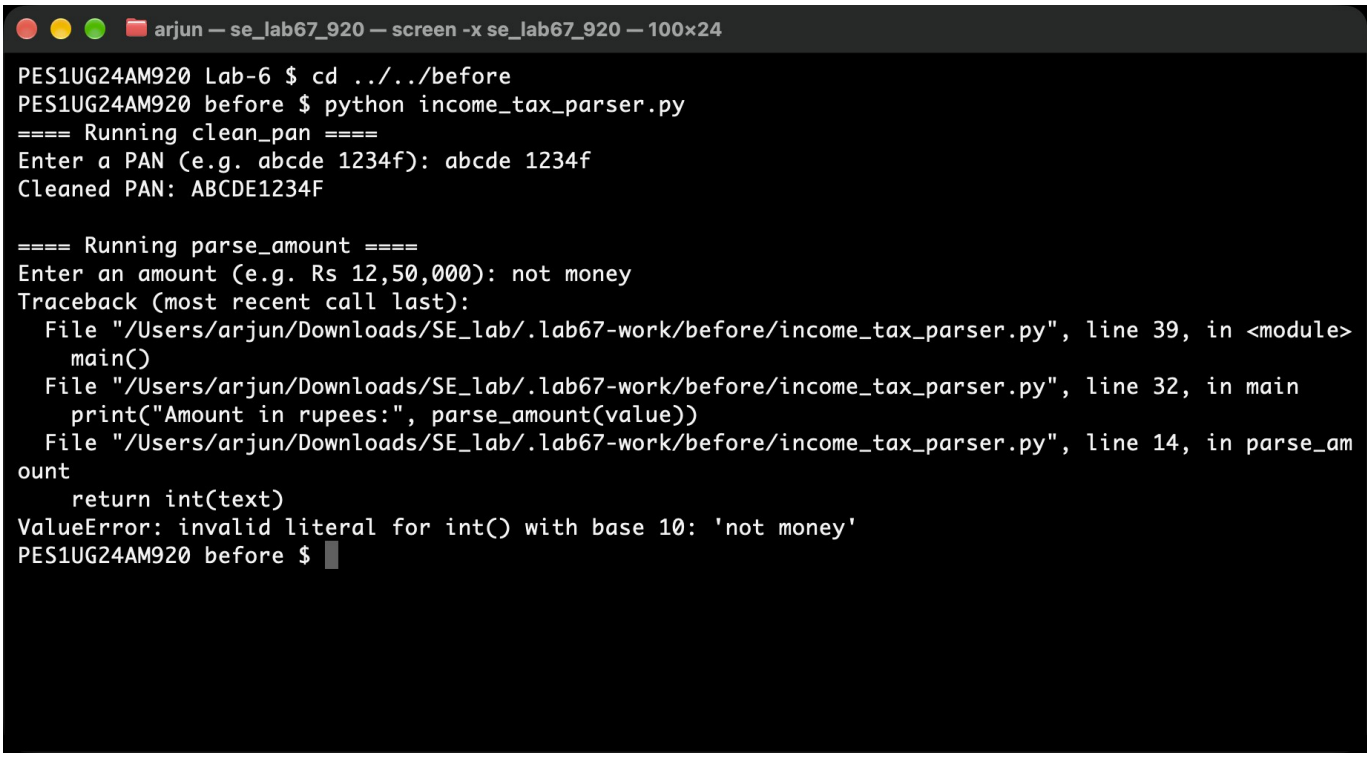


Figure 1 (SS1). Original application fails on an invalid amount.

2. Fuzz Tests Against the Original Program

The completed suite tests all three parsers with Hypothesis and fixed regression cases. Running the same 29 tests against the original program gives 19 failures and 10 passes.

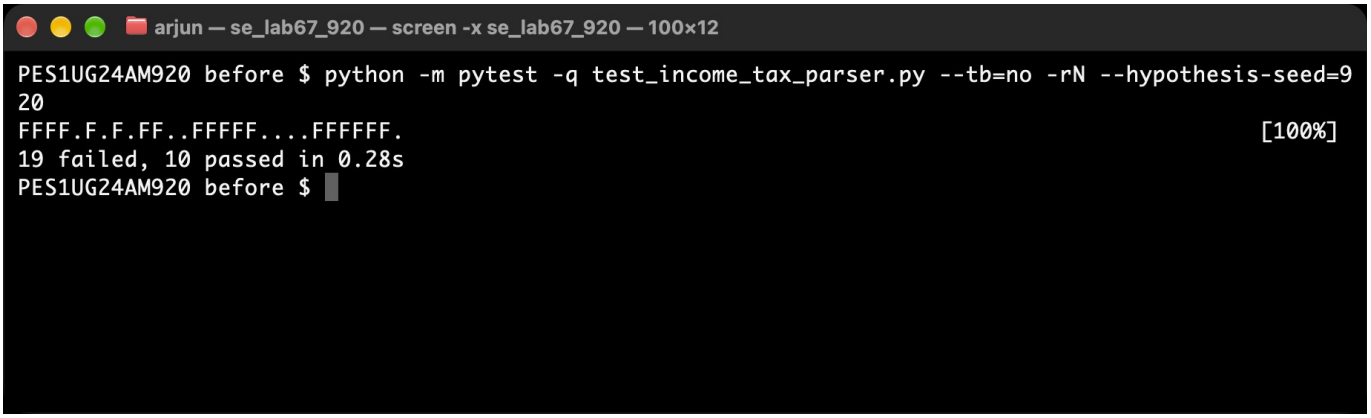


Figure 2 (SS2). Full test suite before correction; tracebacks are suppressed in this summary.

3. Failure Analysis

This focused rerun shows the failure for None in clean_pan(). Hypothesis reports it as an explicit example supplied by @example(None); it is a reproducible regression input, not a randomly discovered value.

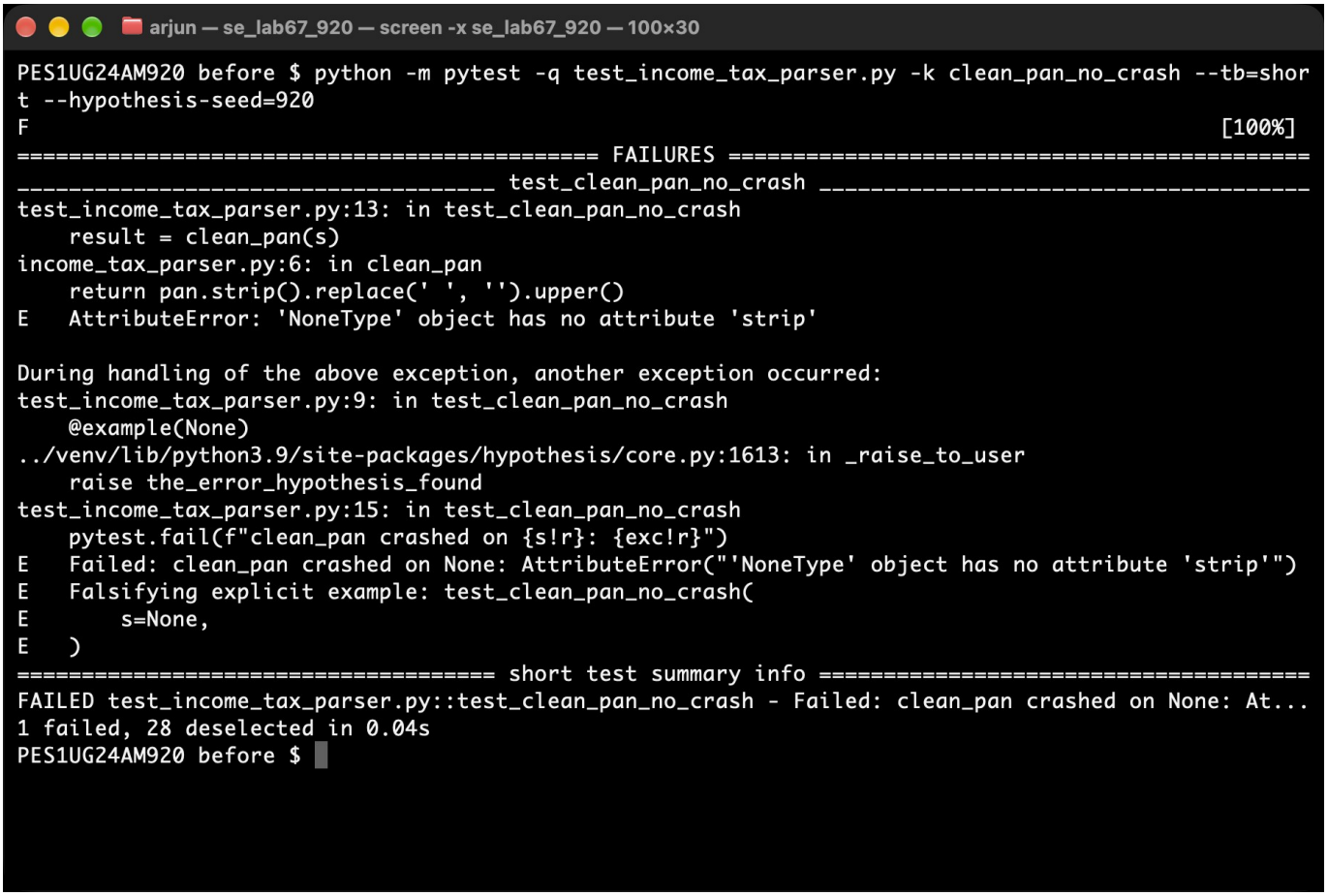


Figure 3. Focused failing property test showing the input, exception and assertion.

Function	Original problem	Correction
clean_pan()	Non-string inputs raise AttributeError.	Return an empty string for non-strings; normalize whitespace and case.
parse_amount()	Invalid values raise exceptions; comma removal accepts malformed grouping.	Check type, length, currency prefix and digit grouping before conversion; return None for invalid values.
parse_tax_slab()	Missing separators raise exceptions; reversed limits and rates above 100 are accepted.	Check syntax, nonnegative ordered bounds and rate 0-100; return None for invalid slabs.

The no-crash properties use Unicode strings and None. Additional properties test non-string values and valid round trips. Fixed examples check known malformed formats. clean_pan() normalizes text; it does not validate the legal PAN structure.

4. Corrected Application Execution

The corrected script accepts the same sample PAN and invalid amount, then continues to reject a malformed tax slab without an unhandled exception.

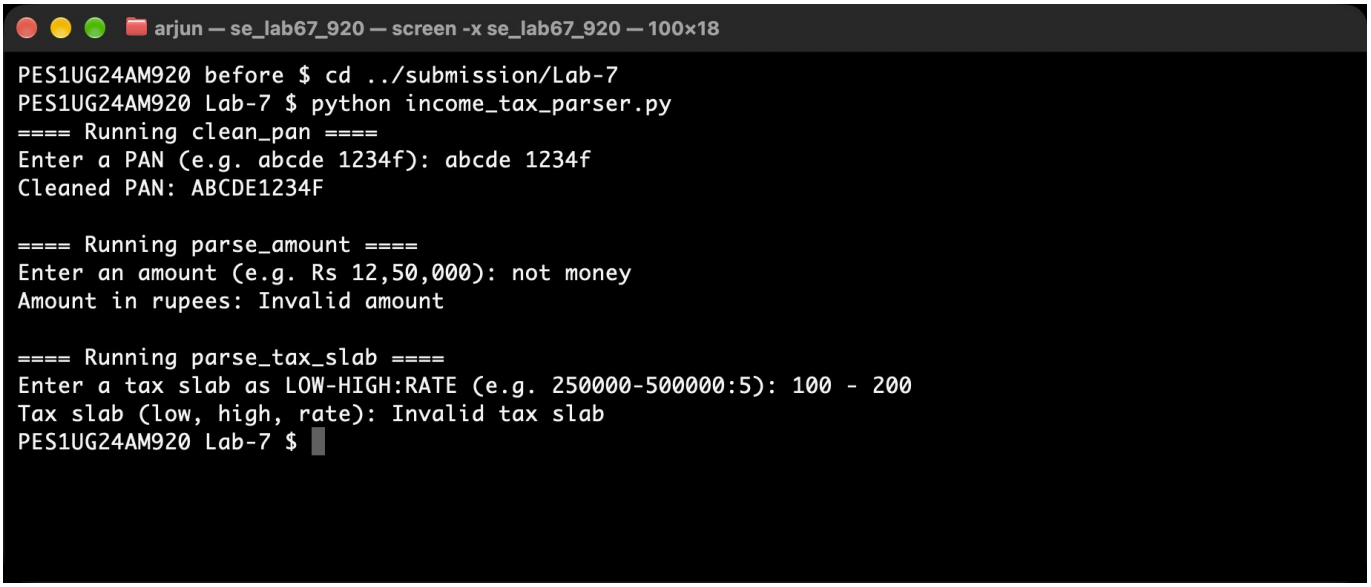


Figure 4 (SS3). Fixed application handles invalid amount and tax slab input.

5. Tests After Correction

The identical test file is rerun against the corrected module with the same Hypothesis seed. All 29 tests pass.

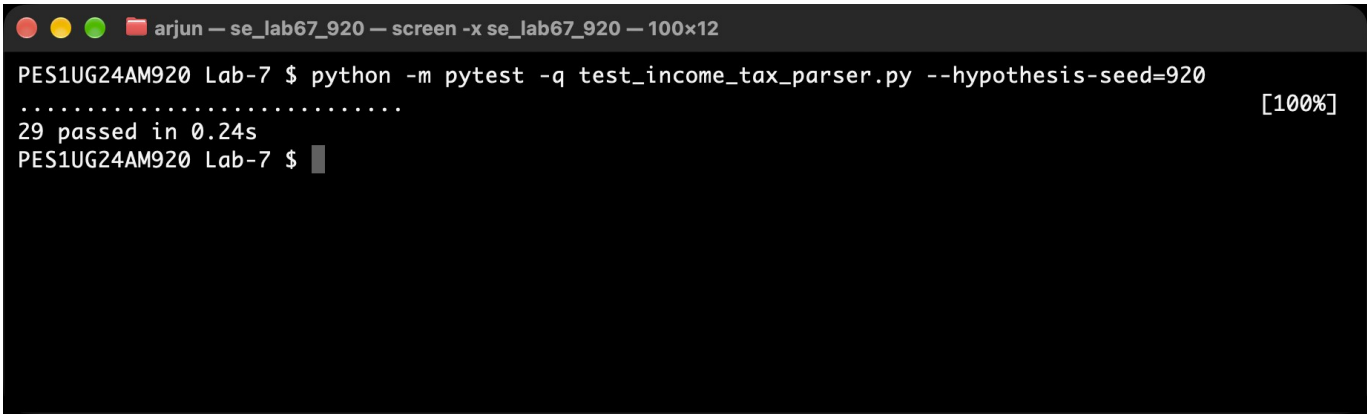


Figure 5 (SS4). Complete fuzz and regression suite after correction.

6. Reflection

What problems were identified? Missing values and unexpected input types caused crashes. Regression checks also exposed malformed amount grouping, reversed slab bounds and rates above 100.

How did fuzzing improve the tests? Generated Unicode strings and non-string values broadened the input space beyond the fixed examples. Explicit None examples make the known crashes reproducible; round-trip properties check valid inputs.

What changes prevented the failures? Type and format checks run before string operations or integer conversion. Invalid amounts and slabs return None, and non-string PAN inputs return an empty string.